



CS-ELEC2C: MACHINE LEARNING

Training Your Multilayer Perceptron

Submitted by:

Jan Vincent G. Elleazar

3CSF

Submitted to:

Jessie James P. Suarez

Table of Contents

A. Effect of Neurons.....	3
B. Effect of Layers.....	4
C. Loss Functions.....	7
D. Layer-Specific Loss.....	7
E. Training Loop.....	9
F. Evaluation Metrics.....	9
G. Learning Rate Impact.....	10
H. Training Time:.....	12

A. Effect of Neurons

Four experiments are conducted to determine the performance of the neural network when more neurons are added to the hidden layer. The number of hidden layers is 1 as a baseline.

TABLE I
EVALUATION METRICS OF MLP BASED ON DIFFERENT NUMBER OF NEURONS IN THE HIDDEN LAYER

Experiments	Training Accuracy	Test Accuracy
Given	0.8433	0.8463
First	0.8434	0.8474
Second	0.8468	0.8523
Third	0.8466	0.8506
Fourth	0.8466	0.8492

On first and second experiments, there are observable improvements in the accuracy of the neural network given that the first experiment used 25 neurons while the second experiment used 50 neurons. However, on the third and fourth experiments, there are no longer improvements in the accuracy of the neural network given that the third experiment used 75 neurons while the fourth experiment used 100 neurons.

TABLE II
TRAINING LOSS OF MLP BASED ON DIFFERENT NUMBER OF NEURONS IN THE HIDDEN LAYER

Experiments	Training Loss
Given	0.3343
First	0.3331
Second	0.3278
Third	0.3292
Fourth	0.3277

Training loss is also observed and it showed that there are small improvements in the first, second, and fourth experiments, On the third experiment however, there is no significant improvement observed.

TABLE III
TRAINING TIME OF MLP BASED ON DIFFERENT NUMBER OF NEURONS IN THE HIDDEN LAYER

Experiments	Training Time (in seconds)
Given	1.5
First	2.8
Second	4.7
Third	6.5
Fourth	7.1

It is also observed that the training time is longer when more neurons are added to the hidden layer.

B. Effect of Layers

Three experiments are conducted to determine the performance of the neural network when more layers are added to the architecture. The given number of hidden layers is 1, the first experiment has 3 hidden layers added, second experiment has 5 hidden layers added, and third experiment has 7 hidden layers added. The number of neurons for each hidden layer is 10 as a baseline.

TABLE IV
EVALUATION METRICS OF MLP BASED ON DIFFERENT NUMBER OF HIDDEN LAYERS IN THE ARCHITECTURE

Experiments	Training Accuracy	Test Accuracy
Given	0.8433	0.8463
First	0.8450	0.8497
Second	0.8460	0.8494
Third	0.7593	0.7588

It is observed that there are small improvements in training accuracy for the first and second experiments while there is a small improvement in test accuracy for the first experiment only. This showed that there is a certain range of hidden layers that will affect the accuracy of the MLP. In this case, 3 hidden layers added will have small improvements while 5 and 7 hidden layers added will have small to no improvements in training or test accuracy.

TABLE V
TRAINING LOSS OF MLP BASED ON DIFFERENT NUMBER OF HIDDEN LAYERS IN THE ARCHITECTURE

Experiments	Training Loss
Given	0.3343
First	0.3298
Second	0.3288
Third	0.5519

In training loss, there is a significant improvement in the first experiment while there is a small improvement in the second experiment. However, there is no improvement in the third experiment. It showed that 7 hidden layers will result in a huge training loss compared to 5 hidden layers added.

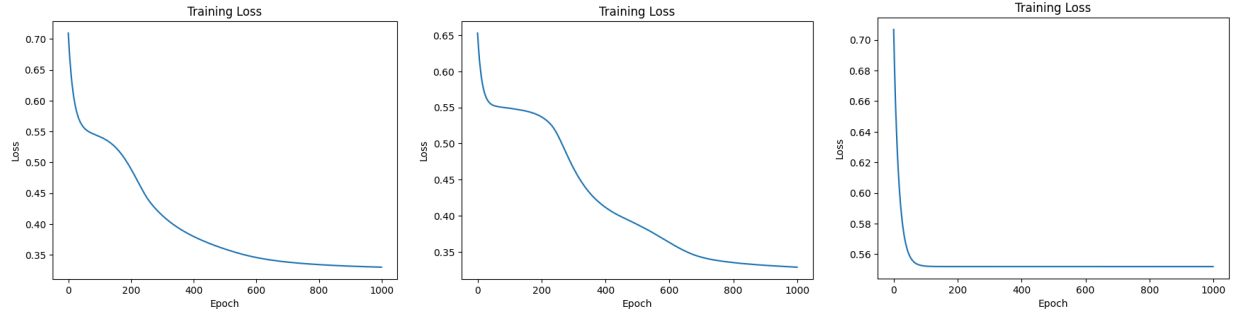


Fig. 1 From left to right: Training loss plot of first, second, and third experiments.

It is also observed in training loss that in the first and second experiments, there is a small retain of loss in between epochs 100 to 200. However on the third experiment, training loss is consistent starting from the 100th epoch. This showed that the model with 3 and 5 hidden layers behaves differently in between epochs 100 to 200 wherein reducing training loss is not consistent in those epochs.

TABLE VI
TRAINING TIME OF MLP BASED ON DIFFERENT NUMBER OF HIDDEN LAYERS IN THE ARCHITECTURE

Experiments	Training Time (in seconds)
Given	1.5
First	3.4
Second	5.0
Third	6.7

Training Time is also observed and it showed that the more hidden layers are added to the architecture, the longer the training time.

C. Loss Functions

Aside from the given BCELoss function, BCEWithLogitsLoss is also appropriate to be used for the problem. BCEWithLogitsLoss combines sigmoid function and BCE into a single function and according to PyTorch, it is more numerically stable than using a plain sigmoid followed by a BCELoss. The use of BCEWithLogitsLoss is demonstrated in the Jupyter Notebook file alongside this report.

D. Layer-Specific Loss

Different loss functions can also be used per layer. However, it would require multiple steps to integrate it.

```
class MultiLossMLP(nn.Module):
    def __init__(self, input_dim, hidden_dim1, hidden_dim2, output_dim):
        super(MultiLossMLP, self).__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim1)
        self.relu1 = nn.ReLU()
        self.layer2 = nn.Linear(hidden_dim1, hidden_dim2)
        self.relu2 = nn.ReLU()
        self.layer3 = nn.Linear(hidden_dim2, output_dim)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        layer1_out = self.relu1(self.layer1(x))
        layer2_out = self.relu2(self.layer2(layer1_out))
        final_out = self.sigmoid(self.layer3(layer2_out))
        return final_out, layer1_out, layer2_out #return intermediate layers.
```

Fig. 2 Code snippet on modified forward pass to return intermediate outputs

First, intermediate outputs during forward pass need to be returned in addition to the final output.

```
# Example loss functions
final_loss_fn = nn.BCELoss()
```

```

layer1_loss_fn = nn.MSELoss() # example layer loss.
layer2_loss_fn = nn.L1Loss() # example layer loss.

# Example data
input_data = torch.randn(32, input_dim)
target_final = torch.randint(0, 2, (32, output_dim)).float()
target_layer1 = torch.randn(32, hidden_dim1) #example target data for layer 1.
target_layer2 = torch.randn(32, hidden_dim2) #example target data for layer 2.

# Forward pass and loss calculation
final_output, layer1_output, layer2_output = model(input_data)

final_loss = final_loss_fn(final_output, target_final)
layer1_loss = layer1_loss_fn(layer1_output, target_layer1)
layer2_loss = layer2_loss_fn(layer2_output, target_layer2)

```

Fig. 3 Code snippet on layer-specific loss implementation

Second, different loss functions are defined for each layer. The loss of each layer will be calculated using its corresponding output and any relevant target data.

```

# Weighted combined loss
total_loss = final_loss + 0.1 * layer1_loss + 0.05 * layer2_loss

```

Fig. 4 Code snippet on weighted combination

Third, the layer-specific losses with the final output loss are combined using weighted sums.

```

# Backward pass
optimizer.zero_grad()
total_loss.backward()
optimizer.step()

```

Fig. 5 Code snippet on backward pass

Last, implement back propagation as given.

E. Training Loop

Forward pass and back propagation are demonstrated in the training loop of the model. It is given in Figure 5 the back propagation of the training loop.

```
for epoch in range(epochs):  
    # Forward pass  
    outputs = mlp(X_train_tensor)  
    loss = criterion(outputs, y_train_tensor)  
    losses.append(loss.item())
```

Fig. 6 Code snippet on forward pass

Figure 6 shows the part of the training loop that handles the forward pass.

F. Evaluation Metrics

Aside from Accuracy, other metrics such as Precision, Recall, F1-score, AUC-ROC, and Confusion Matrix can be used as evaluation metrics for the problem due to the problem's classification nature.

Testing Set Evaluation	Training Set Evaluation
Accuracy: 0.8400	Accuracy: 0.8378
Precision: 0.7381	Precision: 0.7318
Recall: 0.5220	Recall: 0.5148
F1-Score: 0.6115	F1-Score: 0.6044
AUC-ROC: 0.8931	AUC-ROC: 0.8894
Confusion Matrix:	Confusion Matrix:
[[4651 291]	[[18595 1183]
[751 820]]	[3042 3228]]

Fig. 7 Evaluation Metrics of MLP

Among the listed metrics, AUC-ROC and F1-score are most appropriate for this problem due to the Adult Income Dataset being imbalanced. However, Precision, Recall, and Confusion Matrix can still be used to determine and analyze the false positives and false negatives.

G. Learning Rate Impact

Five experiments are conducted to determine the impact of learning rate on the performance of the neural network. Out of the five experiments, three experiments used smaller learning rates (first, second, and third experiments) while the other two experiments used bigger learning rates (fourth and fifth experiments).

TABLE VII
EVALUATION METRICS OF MLP BASED ON DIFFERENT LEARNING RATES

Experiments	Training Accuracy	Test Accuracy
Given	0.8433	0.8463
First	0.7607	0.7603
Second	0.7586	0.7585
Third	0.5448	0.5406
Fourth	0.8494	0.8514
Fifth	0.8328	0.8339

The first, second, and third experiments, given having smaller learning rates of 0.01, 0.001, and 0.0001 respectively, showed no improvement on training and test accuracy. On the fifth experiment with a bigger learning rate of 10, showed no improvement as well on training and test accuracy. On the other hand, the fourth experiment with a bigger learning rate of 1 showed small improvement on training accuracy and a bit significant improvement on test accuracy.

TABLE VIII
TRAINING LOSS OF MLP BASED ON DIFFERENT LEARNING RATES

Experiments	Training Loss
Given	0.3343
First	0.4473
Second	0.6133
Third	0.6943
Fourth	0.3209
Fifth	0.3367

Training Loss is also observed and it showed that the smaller the learning rate, the less improvement on the training loss as seen on the first, second, and third experiments. On the other hand, the fourth experiment showed significant improvement in training loss while the fifth experiment showed no improvement.

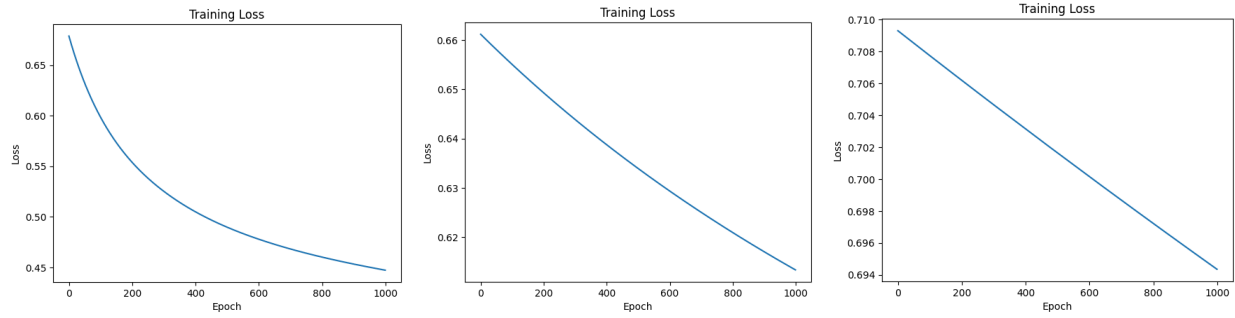


Fig. 8 From left to right: Training loss plot of first, second, and third experiments.

It is also observed that the training loss becomes more linear throughout each epoch when the learning rate becomes smaller.

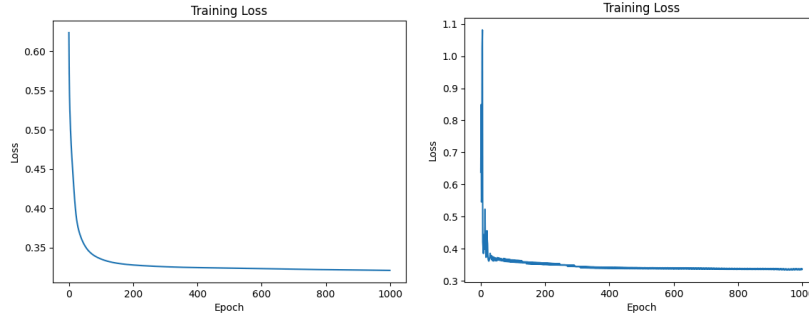


Fig. 9 From left to right: Training loss plot of fourth and fifth experiments

It is also observed that the training loss of the fifth experiment becomes crazy at the beginning of epochs compared to the fourth experiment wherein the training loss is smooth. This showed that training loss starts to behave differently when the learning rate is bigger.

TABLE IX
TRAINING TIME OF MLP BASED ON DIFFERENT LEARNING RATES

Experiments	Training Time
Given	1.5
First	1.8
Second	1.8
Third	1.9
Fourth	1.9
Fifth	2.0

Different learning rates do not significantly affect the training time whether the learning rate is smaller or bigger, as seen on the table above.

H. Training Time:

Based on Table III, training time is longer when more neurons are added to the hidden layer. On Table VI, training time is longer when more layers are added to the architecture. On Table IX, the number of learning rates showed little to no significant effect to the training time.